

Technical Documentation: Architectural Patterns in FleetOps

1. Singleton Pattern

Theoretical Description

The **Singleton Pattern** is a creational design pattern that ensures a class has only one instance while providing a global point of access to that instance. In the context of the Laravel framework, this is managed via the **Service Container**, where an object is instantiated only once during the application's lifecycle and reused for all subsequent requests within the same process.

Why It Was Used

For a logistics and fleet management system like **FleetOps**, performance and resource management are critical. Many components—such as **Repositories** and **Services**—do not need to be re-instantiated every time they are called.

- **Efficiency:** Reducing memory overhead by preventing redundant object creation.
- **Consistency:** Ensuring that stateful services (like logging or real-time tracking) maintain a consistent internal state throughout a single request's execution.
- **Dependency Management:** Simplifies the injection of shared dependencies across various modules.

How It Was Implemented in the Project

In FleetOps, the Singleton pattern is centrally managed within the `App\Providers\ModuleServiceProvider`. This provider uses the `$this->app->singleton()` method to bind specific class interfaces to their implementations. This ensures that whenever a Service or Repository is requested via dependency injection, the container returns the same instance.

Practical Example from the Project

As seen in `app/Providers/ModuleServiceProvider.php`, all major repositories and services are registered as singletons:

```
// File: app/Providers/ModuleServiceProvider.php

// Binding the UserRepository as a Singleton
$this->app->singleton(
    \App\Modules\AuthIdentity\Repositories\UserRepository::class,
    fn() => new \App\Modules\AuthIdentity\Repositories\UserRepository(
        new \App\Modules\AuthIdentity\Models\User()
    )
);

// Binding the AuthService as a Singleton with its dependencies
$this->app->singleton(
    \App\Modules\AuthIdentity\Services\AuthService::class,
```

```
fn($app) => new \App\Modules\AuthIdentity\Services\AuthService(  
    $app-  
>make(\App\Modules\AuthIdentity\Repositories\UserRepository::class),  
    $app->make(\App\Modules\LoggingAudit\Services\LogService::class),  
    $app->make(\App\Modules\LoggingAudit\Services\AuditService::class)  
)  
);
```

Benefits Achieved

- **Scalability:** Reduces the CPU and memory cost per request, allowing the system to handle more concurrent users.
- **Maintainability:** Centralizing instantiation logic in one provider makes it easy to swap implementations or add logging/profiling to object creation.
- **Code Organization:** Decouples the "creation" of objects from their "usage" in controllers.

2. Modular Monolithic Architecture

Theoretical Description

A **Modular Monolithic Architecture** is a design approach where the application is built as a single deployment unit (monolith) but is logically partitioned into independent, loosely coupled modules. Each module represents a distinct business domain and contains its own internal logic, models, and routes.

Why It Was Used

Logistics systems are inherently complex, involving diverse domains like **Route Dispatch**, **Maintenance**, **Real-time Tracking**, and **Order Management**.

- **Isolation:** Changes in the **Maintenance** module are unlikely to break **OrderManagement**.
- **Domain Focus:** Allows developers to focus on a specific business context without navigating the entire codebase.
- **Future-Proofing:** If a specific module (e.g., **RealtimeTracking**) needs to scale independently, it can be extracted into a Microservice with minimal refactoring.

How It Was Implemented in the Project

The project is structured under the **app/Modules** directory. Each module acts as a "mini-application" with the following sub-directories:

- **Controllers/**: Entry points for API requests.
- **Models/**: Database schemas and relations.
- **Services/**: Core business logic.
- **Repositories/**: Data access layer.
- **routes.php**: Module-specific endpoint definitions.

The **ModuleServiceProvider** dynamically loads these modules' routes using the **glob** function, ensuring that adding a new module requires zero configuration in the core application.

Practical Example from the Project

The **OrderManagement** module demonstrates this isolation:

- **Path:** `app/Modules/OrderManagement/`
- **Internal Structure:**
 - `Controllers/InspectionController.php`
 - `Models/Order.php`
 - `Services/OrderService.php`
 - `routes.php` (contains `/api/v1/orders/...` routes)

Benefits Achieved

- **Modularity:** High cohesion within modules and low coupling between them.
 - **Maintainability:** Bugs are easier to locate within the specific module directory.
 - **Team Scalability:** Different teams can work on different modules (e.g., Team A on **AuthIdentity**, Team B on **RouteDispatch**) with minimal merge conflicts.
-

3. MVC Architecture (Model-View-Controller)

Theoretical Description

MVC is an architectural pattern that separates an application into three main logical components:

1. **Model:** Manages data and business rules.
2. **View:** Handles presentation (in FleetOps, this is represented by JSON API responses).
3. **Controller:** Intermediary that handles user input and updates the model/view.

FleetOps extends this with the **Service-Repository** pattern for better separation.

Why It Was Used

Using MVC provides a standard, predictable structure for the FleetOps RESTful API.

- **Separation of Concerns:** Prevents business logic from leaking into the database schema or the HTTP handling layer.
- **Reusability:** Business logic inside **Services** can be reused by both the Web API and scheduled Cron jobs (Console commands).

How It Was Implemented in the Project

- **Model:** Located in `app/Modules/{ModuleName}/Models/`. These utilize Eloquent ORM for database interaction.
- **Controller:** Located in `app/Modules/{ModuleName}/Controllers/`. These handle HTTP requests, validate input via **FormRequests**, and call Services.
- **View (API):** Since FleetOps is a backend system, the "View" is the JSON payload returned by controllers, often formatted using Laravel's response helpers.

Practical Example from the Project

Consider the **Inspection** workflow:

- 1. **Controller:** `InspectionController` receives a POST request to store a pre-trip inspection.
- 2. **Service:** `InspectionService` contains the logic to validate if the vehicle is eligible for inspection.
- 3. **Model:** `PreTripInspection` model stores the data in the database.
- 4. **Response (View):** A JSON response confirming the successful creation.

```
// Controller logic example
public function store(StoreInspectionRequest $request)
{
    // Controller calls the Service (Business Logic)
    $inspection = $this->inspectionService->create($request->validated());

    // Returns the "View" (JSON Response)
    return response()->json(['data' => $inspection], 201);
}
```

Benefits Achieved

- **Code Organization:** Every class has a single, clear responsibility.
- **Reusability:** The `OrderRepository` can be used by the `OrderService`, the `RouteService`, and the `ReportingService` without duplicating SQL queries.
- **Testability:** Services and Repositories can be unit-tested in isolation from the HTTP layer.

Summary of Improvements

Feature	Singleton	Modular Monolith	MVC / Service-Repo
Scalability	Low memory footprint	Domain-based scaling	Efficient logic reuse
Maintainability	Centralized instantiation	Isolated code domains	Clean separation of concerns
Modularity	Decoupled creation	Strict domain boundaries	Interface-based design
Organization	Standardized providers	Highly organized <code>app/Modules</code>	Predictable file structure